

Implementação de Ordenação de Dados com Bucket Sort

Aluno: André Paulo Streck Renner

Professor: Antônio Marcos de Oliveira Candia

25/04/2025

Universidade Federal de Santa Maria



Resumo:

O trabalho visa a criação de um algoritmo híbrido de ordenação com o Bucket Sort. Dentre os métodos de ordenação utilizados o que mais se destacou foi o com o Quicksort, todavia, o algoritmo tem qualidades e defeitos que não o torna totalmente superior aos demais.

Introdução:

O trabalho em questão visa consolidar o conhecimento em técnicas de ordenação de dados, o qual propunha implementar um algoritmo híbrido de ordenação com base no Bucket Sort. Tal algoritmo de ordenação é um método que distribui os dados em múltiplos "baldes" (buckets), de acordo com intervalos específicos, ordenando cada balde individualmente ao uso de outro algoritmo auxiliar. Ele é especialmente eficiente para dados uniformemente distribuídos, alcançando complexidade média de $O(n + k)$, onde n é o número de elementos e k o número de baldes, mas pode chegar a $O(n^2)$ no pior caso. Logo, a proposta era criar um Bucket Sort com, no mínimo, 3 variações para ordenação em cada balde. E, desse modo, comparar o desempenho na ordenação das variações selecionadas com o método padrão (Insertion Sort em cada bucket) com 100 mil, 1 milhão e 100 milhões de valores aleatórios.

Metodologia:

Primeiramente, dispus-me a pesquisar no material fornecido no moodle sobre os diferentes métodos de ordenação de dados, analisando suas peculiaridades, a fim de selecionar algoritmos que mais se aproximem do ideal em determinadas ocasiões. Após isso, escolhi os algoritmos Quicksort, Heap Sort e Merge Sort, já que dispunham de ótima eficiência em ocasiões gerais. Além disso, optei por determinar o número de buckets n por sua raiz quadrada de n , assim escolhida como heurística por proporcionar um equilíbrio entre número de buckets e elementos por bucket, minimizando o custo de inserção e ordenação interna. Por fim, determinei que a quantidade de posições dentro de cada bucket alocada e realocada dinamicamente conforme necessidade.

Características dos Algoritmos Utilizados:

Insertion Sort:

Algoritmo de ordenação incremental que opera construindo, passo a passo, uma sequência ordenada a partir do início do conjunto, inserindo cada novo elemento na posição adequada em relação aos já ordenados. Trata-se de um algoritmo **estável** (preserva a ordem relativa de valores

equivalentes) e in-place (não demanda memória adicional significativa), com complexidade $O(n^2)$ no pior caso, porém comportando-se de forma eficiente para conjuntos pequenos ou quase ordenados, nos quais atinge desempenho próximo a $O(n)$. Sua simplicidade e baixo overhead o tornam uma escolha adequada para vetores curtos ou como base de ordenações híbridas.

Quicksort:

Algoritmo de divisão e conquista que trabalha com a seleção de um pivô qualquer para dividir o conjunto de valores entre maiores e menores, sendo esses subconjuntos novamente selecionados novos pivôs, até todos elementos tornarem-se pivôs. É um algoritmo **não-estável** (não mantém a ordem inicial de valores iguais) com complexidade $O(n \log(n))$, **in-place** (não utiliza memória extra), todavia degenera com a seleção de pivôs desbalanceados. Utilizei da seleção aleatória de um pivô, já que, assim, há maior probabilidade de uma boa escolha de pivô e maior segurança para a aplicação do método de ordenação.

Heap Sort:

Algoritmo de ordenação baseado na estrutura de dados Heap (geralmente um Heap Máximo), que organiza os elementos de forma hierárquica. O maior valor (raiz) é trocado com o último elemento, e o heap é reajustado (heapify) para manter sua propriedade. O processo se repete até o heap conter apenas um elemento. É um algoritmo **não-estável** (não preserva a ordem de valores iguais), com complexidade $O(n \log(n))$ em todos os casos, **in-place** (utiliza memória constante), e não sofre degradação com a entrada, sendo eficiente mesmo em casos desfavoráveis.

Merge Sort:

Algoritmo de divisão e conquista que divide recursivamente o conjunto de valores em metades até obter subconjuntos unitários, realizando em seguida a etapa de mesclagem ordenada desses subconjuntos. É um algoritmo estável (mantém a ordem inicial de valores iguais), com complexidade $O(n \log(n))$ mesmo nos piores casos. **Não é in-place**, pois utiliza memória extra proporcional ao tamanho da entrada para armazenar temporariamente os elementos durante a mesclagem. Ideal para grandes volumes de dados e cenários onde a estabilidade é importante.

Resultados:

Dessa forma, implementei os algoritmos híbridos e realizei diversos testes com as entradas previamente listadas e obtive o seguinte apuramento:

Tempo de Execução Médio						
Método de Ordenação	Dados Aleatórios			Parcialmente Ordenados		
	10^5	10^6	10^8	10^5	10^6	10^8
Insertion Sort	95 ms	815 ms	6 minutos	96 ms	586 ms	4 minutos
Quicksort	70 ms	383 ms	51 s	78 ms	527 ms	2 minutos
Heap Sort	83 ms	487 ms	45 s	98 ms	558 ms	59 s
Merge Sort	94 ms	572 ms	51 s	101 ms	592 ms	56 s

Discussão:

Após estes testes, concluí:

- Para ordenações genéricas, com elevada quantidade de elementos e que não necessitam de estabilidade, o Bucket Sort com o Quicksort é superior aos demais utilizados, apenas degenerando em casos de má escolha de pivôs ($O(n^2)$), como, por exemplo, no caso de 10 milhões de elementos parcialmente ordenados;
- Para ordenações estáveis e com muitos valores, o Bucket Sort com Merge Sort é superior aos demais métodos de ordenação, já que utiliza divisão e conquista para ser eficiente e mantém sua complexidade constante ($O(n \log n)$). Além de não degenerar em nenhum caso;
- O Bucket Sort com Heap Sort é um bom algoritmo para ordenação de muitos dados, pois utiliza a construção de uma heap para ser ágil, fato este que o torna inferior em casos com menos valores, já que construí-la é um processo custoso.
- O Bucket Sort com Insertion Sort é útil apenas para uma pequena quantidade de dados ou se estes já estão parcialmente ordenados. Uma boa estratégia é aumentar o número de buckets para o algoritmo possuir menos elementos para processar e assim ser eficiente.

Conclusão:

Em suma, a realização deste trabalho permitiu uma análise comparativa entre diferentes algoritmos de ordenação aplicados ao Bucket Sort, evidenciando, assim, as forças e limitações de cada abordagem híbrida. O Quicksort, quando utilizado dentro dos buckets, demonstrou ser o mais eficiente para grandes quantidades de dados aleatórios, devido à sua complexidade média e à agilidade na divisão dos subconjuntos. No entanto, mostrou-se vulnerável à degradação de desempenho em cenários de pivôs desbalanceados.

O Merge Sort, por sua vez, destacou-se por sua estabilidade e comportamento previsível, mantendo uma performance sólida mesmo em casos extremos, o que o torna ideal para aplicações que demandam ordenação estável. Já o Heap Sort apresentou bom desempenho com grandes volumes de dados, mas sua eficiência caiu em vetores menores, devido ao custo da construção da heap. O Insertion Sort provou ser uma escolha válida apenas para conjuntos pequenos ou parcialmente ordenados, sendo simples e leve.

Conclui-se, portanto, que a performance do Bucket Sort depende diretamente da escolha do algoritmo auxiliar, da distribuição dos dados e do volume da entrada, reforçando a importância da análise contextual na seleção de estratégias de ordenação.

Algoritmo com Bucket Sort	Melhor ocasião	Complexidade	Estável	Pior desempenho
Insertion Sort	Poucos dados ou quase ordenados	$O(n^2/k)$	Sim	$O(n^2)$
Quicksort	Grande volume de dados aleatórios não-estáveis	$O(n \log(n/k))$	Não	$O(n^2)$
Heap Sort	Grande volume de dados, com qualquer entrada	$O(n \log(n/k))$	Não	$O(n \log n)$
Merge Sort	Grande volume de dados que pedem estabilidade	$O(n \log(n/k))$	Sim	$O(n \log n)$

Material Bibliográfico:

- **VIANA, Geraldo Waldir Rodrigues; GARCIA, Valdério Rodrigues; NOBRE, Ricardo Holanda.** *Computação: Pesquisa e Ordenação de Dados*. 1. ed. Fortaleza: EdUece, 2015.
- **KNUTH, Donald Ervin.** *The Art of Computer Programming: Volume 3 – Sorting and Searching*. 2. ed. Boston: Addison-Wesley, 1998.